Aula 4: Estrutura e Níveis de Teste

Descrição: Nesta aula vamos exibir uma visão geral sobre as camadas de teste e a pirâmide de testes como guia de eficiência. Demonstraremos os níveis de teste: Unitário, Integrado, de Sistema e de Aceitação. Também das diferenças entre Testes Funcionais (o que o sistema faz) e Não Funcionais (como o sistema se comporta).

1. Abertura da Aula

Hoje, teremos uma visão geral sobre as camadas de teste e a pirâmide de testes. No mundo real do desenvolvimento de software, não podemos simplesmente terminar de programar e "**ver se funciona**" no final. Precisamos de uma **estrutura organizada** que garanta que cada pequena parte do código se comunique perfeitamente com o todo. O grande problema que resolveremos hoje é: **como organizar nossos testes de forma eficiente, gastando menos tempo e capturando erros o mais cedo possível?**

O desafio de vocês consiste em **evoluir um produto digital funcional** já existente, transformando-o em uma solução tecnológica inovadora para o mercado. Entender a estrutura e os níveis de teste permitirá que **vocês desenvolvam um sistema web funcional e diferenciado**, validado por um **plano de testes rigoroso e consolidado** em uma apresentação técnica para a banca.

---

2. Conceitos Fundamentais

2.1. Níveis de Teste

Os testes de software são divididos em etapas para **isolar problemas e facilitar a correção**. Os principais níveis de teste são: **Unitário, Integrado, de Sistema e de Aceitação**.

- O **Teste Unitário** foca na **menor parte testável do código**, como uma única função, garantindo que ela faça o que deve isoladamente.
- O **Teste Integrado** avalia como essas funções e módulos **interagem entre si** (por exemplo, o sistema salvando algo no banco de dados).
- O **Teste de Sistema** valida o comportamento da **aplicação inteira montada**, de ponta a ponta.
- Por fim, o **Teste de Aceitação** é focado no **usuário final** ou cliente, verificando se o software **resolve o problema de negócio** para o qual foi criado.

Demonstração:
Vamos imaginar o sistema da nossa SA e uma função simples de cálculo de desconto. Aqui está como testaríamos a menor unidade dela (Teste Unitário) em Python:

Python
```python
# A função de negócio (O que o sistema faz)
def calcular_desconto(valor_total, cupom):
    if cupom == "SENAI20":
        return valor_total * 0.80 # 20% de desconto
    return valor_total

# O Teste Unitário (A validação)
def test_calcular_desconto():
    # Prepara os dados
    valor_compra = 100.0

    # Executa a função
    resultado = calcular_desconto(valor_compra, "SENAI20")
```

```
    # Valida o resultado esperado
    assert resultado == 80.0, f"Erro: Esperado 80.0, mas retornou {resultado}"
    print("Teste Unitário passou com sucesso!")

test_calcular_desconto()
```

***Explicação**:* A função `calcular_desconto` é a nossa unidade. O `test_calcular_desconto` é o nosso roteiro de teste automatizado. O comando `assert` verifica se o valor retornado pela função é igual ao que esperamos (80.0). Se for, o teste passa.

Debate:

Se precisássemos testar a função `calcular_desconto` puxando a validade do cupom direto de um Banco de Dados, qual nível de teste estaríamos aplicando?

Exercício:

Agora crie uma função em Python chamada `verificar_maioridade(idade)` que retorne `True` se a idade for maior ou igual a 18, e `False` caso contrário. Em seguida, escreva um **teste unitário** para validar essa função usando a estrutura do exemplo acima.

2.2. Testes Funcionais vs. Não Funcionais

É crucial entender as diferenças entre **Testes Funcionais** (o que o sistema faz) e **Não Funcionais** (como o sistema se comporta).

- Os **Testes Funcionais** validam as regras de negócio:
  - o login funciona?
  - o botão de enviar manda o e-mail?
  - o cálculo do frete está correto?
  - Eles respondem se o sistema executa suas funções corretamente.
- Já os **Testes Não Funcionais** avaliam atributos de qualidade da aplicação:
  - O login suporta 1000 usuários simultâneos?
  - O sistema é seguro contra invasões?
  - Ele carrega em menos de 2 segundos?
  - Ele responde sobre o comportamento e a robustez da aplicação.

Demonstração: Imaginem a tela de login do projeto da empresa de vocês.

- **Cenário Funcional:** Inserir usuário "admin", senha "123", clicar em entrar.
  - *Resultado esperado:* Acesso liberado ao painel.
- **Cenário Não Funcional (Performance):** Simular 500 usuários tentando fazer login exatamente no mesmo segundo.
  - *Resultado esperado:* O servidor não cai e o tempo de resposta permanece abaixo de 3 segundos.

Exercício:

Acessem um site famoso de e-commerce e listem no quadro 2 requisitos funcionais que vocês enxergam na tela e 2 requisitos não funcionais que vocês supõem que existam por trás daquela tela.

Classifiquem as afirmações abaixo como (F) Funcional ou (NF) Não Funcional:

1. ( ) O aplicativo deve permitir o cadastro de novos clientes.
2. ( ) O aplicativo deve estar disponível 99,9% do tempo.
3. ( ) A senha do usuário deve ser armazenada de forma criptografada.
4. ( ) O sistema deve emitir um alerta sonoro quando uma nova mensagem chegar.

2.3. A Pirâmide de Testes

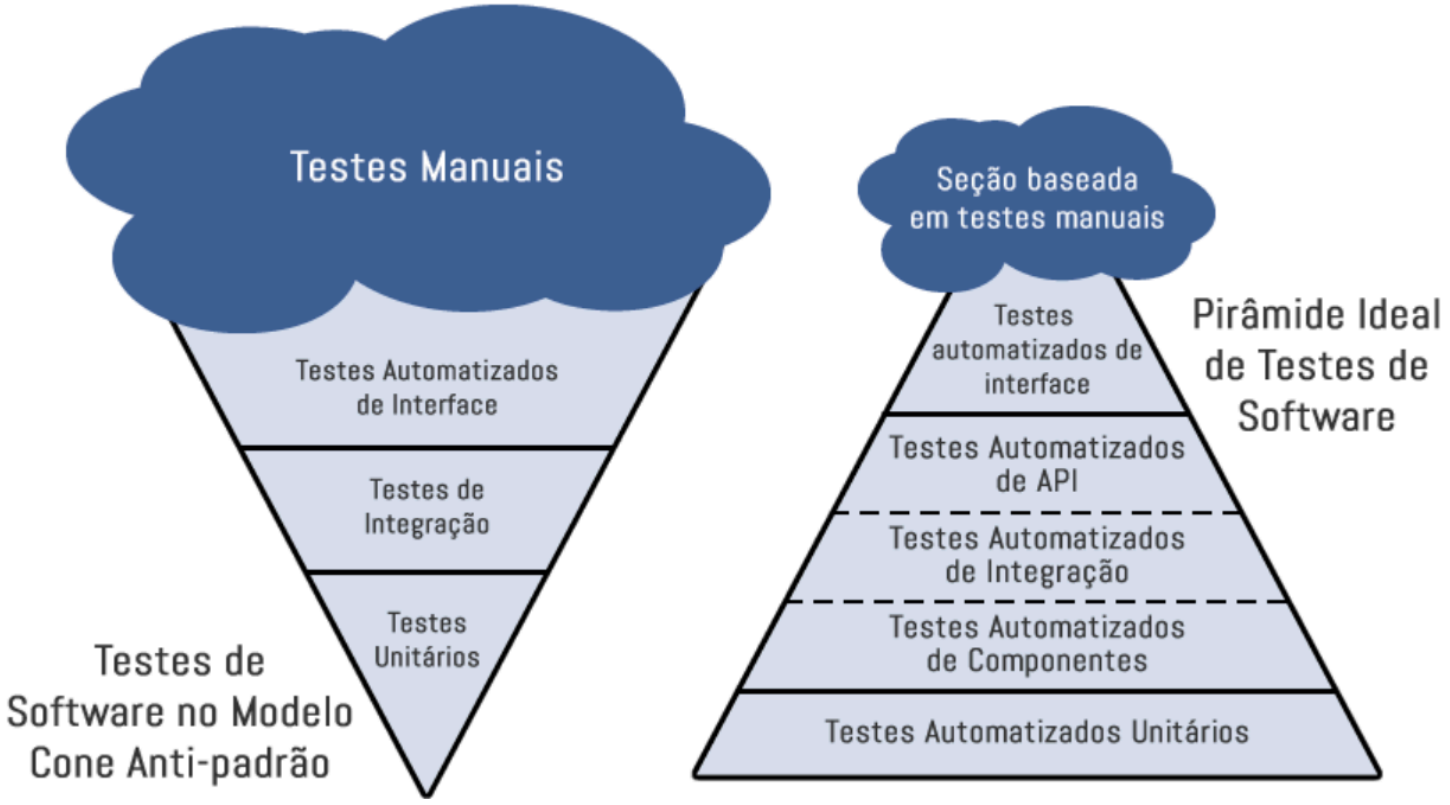
A Pirâmide de Testes serve como **guia de eficiência**. Ela ilustra como devemos distribuir nossos esforços de automação de testes para ter um projeto saudável, rápido e barato de manter.

- A **base** da pirâmide é **larga** e representa os **Testes Unitários**: eles devem ser a grande maioria, pois são rápidos de executar e baratos de criar.
- O **meio** da pirâmide representa os **Testes de Integração**: em menor quantidade que os unitários, pois são um pouco mais lentos e complexos.
- O **topo** representa os **Testes de UI (Interface/Sistema)**: devem ser poucos, apenas cobrindo as jornadas críticas, pois são muito lentos para rodar e difíceis de manter.

Demonstração:
Se a sua equipe da SA tem 100 testes automatizados, a divisão ideal baseada na pirâmide seria algo como:

- **70 Testes Unitários:** Validando cada função matemática, validação de string, etc. Executam em milissegundos.
- **20 Testes de Integração:** Validando a comunicação da API com o Banco de Dados. Executam em alguns segundos.
- **10 Testes de Sistema/UI:** Validando o fluxo de ponta a ponta ("Abrir navegador, logar, comprar, sair"). Executam em vários minutos.

Se a equipe inverter isso (**Anti-padrão do Cone de Sorvete**), o sistema demorará horas para ser testado e a entrega atrasará.



Debate: Por que os testes de Sistema (Interface gráfica) ficam no topo e em menor quantidade na pirâmide de testes?

2.4. Teste de Mesa e Ferramentas Interativas

Antes de automatizar ou mesmo codificar testes complexos, fazemos a prática de **teste de mesa** utilizando ferramentas interativas. O teste de mesa é uma técnica manual onde **simulamos a execução do código** passo a passo, anotando no papel ou em planilhas o valor das variáveis a cada linha

executada. Isso ajuda a encontrar erros de lógica visualmente e estimula o pensamento crítico antes do uso de compiladores.

Demonstração Prática: Acompanharemos a simulação de uma variável `contador` num laço de repetição `while` de 1 a 3. Anotaremos em uma tabela no quadro as mudanças de estado da variável. Em seguida, utilizaremos uma ferramenta interativa como o Kahoot! para jogar um quiz rápido sobre os estados das variáveis apresentadas.

Exercício Guiado:
Execução coletiva de um teste de mesa de uma lógica de autenticação simples.
Exercício de Fixação:
Acessem o Kahoot preparado para a aula e respondam aos 5 desafios de rastreamento de código (teste de mesa virtual).

---

3. Demonstração Aplicada
Vamos consolidar isso em um exemplo real do mercado. Pensem em um sistema de *Delivery*:

1. **Estrutura de Código:** Temos o Front-end (App do usuário), a API (Motor de regras) e o Banco de Dados.

2. **Lógica de Funcionamento:**
   - **Unitário:** Testaremos apenas a função `calcula_taxa_entrega(distancia)`. Passamos 5km, esperamos retornar R$10,00. Rápido, direto.

   - **Integrado:** Testaremos a função `fazer_pedido()`. Ela deve chamar a API de pagamento externa e gravar a transação no nosso Banco de Dados. Testamos se a comunicação entre essas três peças não quebra.

   - **Sistema:** Automatizamos um robô para abrir o site, clicar no hambúrguer, preencher o endereço, finalizar a compra e checar se a tela de "Sucesso" aparece.

3. **Resultado Esperado:** Ao juntar essas camadas seguindo a proporção da pirâmide, a equipe consegue detectar se o problema está na matemática (unitário), na comunicação (integrado) ou na tela (sistema) em questão de minutos, garantindo aderência aos requisitos iniciais.

---

4. Casos Reais do Mercado

- **Nubank e a Base da Pirâmide:** As *fintechs* **dependem criticamente de Testes Unitários**. Imagine um erro de cálculo de juros aplicado a milhões de clientes! O Nubank possui milhares de testes unitários que rodam a cada modificação de código de um desenvolvedor. Se um único teste falhar, o código é bloqueado de ir para a produção.

- **Netflix e Testes Não Funcionais:** A Netflix é pioneira em **testes não funcionais de resiliência** e performance. Eles têm um sistema chamado "**Chaos Monkey**", que desliga servidores de propósito no ambiente de produção para testar (**em nível de sistema e integração**) se a plataforma consegue se curar sozinha sem que o usuário final perceba o filme travando.

---

5. FAQ -- Dúvidas Reais

1. **Posso testar a interface do usuário (UI) usando Testes Unitários?**
   ***Resposta**: **Não é o ideal.** Testes unitários focam no código lógico (**regras de negócio**). O teste de interface (ver se um botão é vermelho) pertence à camada de Testes de Sistema ou de UI (no topo da pirâmide).*

2. **Se eu fizer excelentes Testes de Sistema, preciso fazer Testes Unitários?**
   ***Resposta**: **Sim**! Testes de sistema são lentos e difíceis de rastrear. Se um teste de sistema falha, você sabe que o fluxo quebrou, mas é o teste unitário que aponta exatamente em qual linha de código a matemática falhou.*

3. **Um software pode falhar num teste Não Funcional, mas passar no Funcional?**
   ***Resposta: Perfeitamente**. O seu sistema pode fazer um PIX corretamente (**Funcional Aprovado**), mas demorar 5 minutos processando e consumir 100% do servidor, travando para os outros usuários (**Não Funcional Reprovado**).*

4. **Quem define o que o Teste de Aceitação vai validar?**
   ***Resposta: Geralmente são os usuários finais**, os Product Owners (**PO**) ou **o cliente** que encomendou o software, pois o objetivo é atestar se o software atende ao negócio.*

5. **Quando a gente começa a testar no projeto da SA?**
   ***Resposta: Desde o primeiro dia!** A mentalidade moderna de QA (**Quality Assurance**) adota o **"Shift-Left"**, que significa **trazer os testes o mais cedo possível**, inclusive testando a ideia e os requisitos antes mesmo de codificar.*

---

6. Desafio Prático

Contexto: Para uma a feira de investidores, decidiram implementar uma funcionalidade de "Match Inteligente" na plataforma. Esse algoritmo vai conectar investidores a startups promissoras baseado em *tags* de interesse comum.

Requisitos: Para validar esse aperfeiçoamento, em equipes, vocês devem elaborar um esboço da estratégia de testes em um documento (pode ser digital ou físico). Vocês deverão:

1. Escrever e descrever **2 Testes Funcionais** e **2 Testes Não Funcionais** para o "Match Inteligente".

2. Mapear onde esses testes se encaixam na **Pirâmide de Testes** (Unitário, Integrado, Sistema).

3. Realizar um **Teste de Mesa** da lógica de "Match": crie duas entidades no papel (Investidor A com tags [Tech, IA] e Startup B com tags [IA, Saúde]) e execute visualmente como o código decidiria se há Match.

Resultado Esperado: O grupo deverá apresentar à turma a sua pirâmide preenchida e demonstrar que compreende as camadas, organizando o ambiente para o desenvolvimento das rotinas de testes.

---

7. Conexão com a Situação de Aprendizagem

Todo o conteúdo de hoje visa preparar vocês para **avaliar o resultado obtido no teste** e identificar tipos, função, ferramentas e plano de teste de acordo com a programação de sistemas. No momento da apresentação para a banca , não basta mostrar que o sistema "**parece**" funcionar; vocês terão que

demonstrar a análise documental dos resultados e comprovar que possuem um cronograma de testes estruturado nas camadas corretas da pirâmide, conferindo robustez e confiabilidade à entrega de vocês.

---

8. Preparação para a Próxima Aula

Hoje entendemos ***como* estruturar e *quais* são os níveis de testes**. Mas como sabemos *o que exatamente* deve ser testado? O que o cliente espera de verdade?

Na nossa próxima aula (**Aula 5**), mergulharemos na **leitura e análise de requisitos**. Vamos aprender a dissecar documentos e histórias de usuário para e**xtrair os Critérios de Aceitação** exatos antes mesmo de escrevermos o primeiro teste ou linha de código.